RAG MASTERY SERIES · CLASS 2 OF 6 · AMANAILAB.COM

INDEXING PIPELINE

Class 2: Documents → Chunks → Embeddings → Vector DB

The complete deep dive into how RAG prepares your knowledge base

WHAT YOU WILL LEARN IN THIS CLASS:

- Document Loading — All source types & loaders explained
- Chunking Deep Dive — 7 strategies compared with real examples
- Chunk Size, Overlap & Metadata — Golden rules you must follow
- Embeddings — How text becomes numbers, all 2026 models compared
- Similarity Metrics — Cosine, Dot Product, Euclidean explained
- Vector Databases — ChromaDB, Pinecone, FAISS, Qdrant & more

C1 Foundation	**C2 Indexing**	C3 Retrieval	C4 Generation	C5 Advanced	C6 Production

6-Class RAG Mastery Roadmap — You Are Here: Class 2

Class 2 — Table of Contents

2.1 | Recap: Where Are We?

Quick refresher from Class 1

In Class 1, we learned that RAG has **two phases**. Today we deep-dive into **Phase 1: Indexing** — the process of converting your raw documents into a searchable knowledge base.

The RAG Pipeline — Class 2 Focus

Phase 1: INDEXING (Today's Class)
Documents → Load → Chunk → Embed → Store in Vector DB

Phase 2: QUERYING (Class 3 & 4)
User Question → Embed → Search → Top-K → LLM → Answer

Indexing is the FOUNDATION — if your documents are poorly chunked or badly embedded, no amount of advanced retrieval or prompt engineering will save you.

Why Indexing is 70% of RAG Success

Most RAG failures are NOT because of the LLM or prompt — they're because of bad indexing:
• Wrong chunk size → retrieves irrelevant content
• No chunk overlap → loses context at boundaries
• Bad embedding model → can't find semantically similar content
• No metadata → can't filter or cite sources

Master indexing, and your RAG system is already 70% of the way to production quality.

2.2 | Document Loading

All source types, loaders & what gets extracted

The first step in indexing is **reading your raw documents**. RAG frameworks provide loaders for every format. Each loader converts raw files into **Document objects** containing two things: the extracted text (**page_content**) and metadata like source filename, page number, and creation date.

Supported Source Types

Source Type	What Gets Extracted	Common Loaders (LangChain)	Challenges
PDF Files	Text from each page + page number metadata	PyPDFLoader, PDFPlumber, Docling	Tables, images, scanned PDFs often extracted poorly
Word / DOCX	Paragraphs, headings, lists as text	UnstructuredWordDocumentLoader	Formatting lost, tables may break
Web Pages	Cleaned text from URLs, removing nav/ads	WebBaseLoader, BeautifulSoup, Crawl4AI	Dynamic JS content may not load
CSV / Excel	Rows converted to text or key-value pairs	CSVLoader, ExcelLoader	Large files need row-level chunking
Databases (SQL)	Query results as text documents	SQLDatabaseLoader, BigQueryLoader	Need to define what queries to run
Notion Pages	Page content, blocks, sub-pages	NotionDirectoryLoader	Nested blocks need flattening
Confluence	Wiki pages with headings and content	ConfluenceLoader	Authentication setup needed
Emails	Subject, body, sender, date, attachments	GmailLoader, OutlookLoader	HTML emails need cleaning
Code Files	Source code with file path metadata	DirectoryLoader + TextLoader	Need language-aware chunking
Audio / Video	Transcribed text (via Whisper)	AssemblyAILoader, WhisperLoader	Transcription quality varies

What is a Document Object?

Every loader returns a list of **Document objects**. Each Document has exactly two fields:

Field	What It Contains	Example
page_content	The actual text extracted from the source	'All full-time employees are entitled to 24 days of annual leave per calendar year...'

Field	What It Contains	Example
metadata	Dictionary of source information: file name, page number, URL, date, etc.	{'source': 'hr_policy_2024.pdf', 'page': 12, 'created': '2024-01-15'}

Why Metadata Matters

Metadata is NOT optional — it's essential for:
• Source citation: 'Answer from hr_policy.pdf, Page 12'
• Filtering: Only search documents from a specific category/date
• Deduplication: Avoid indexing the same document twice
• Access control: Different users see different documents

Always add as much metadata as possible: source, page, section heading, date, author, category.

The PDF Problem — Why Document Parsing is Hard

PDFs are the most common document type in enterprise RAG, but they're also the **hardest to parse**. A PDF is designed for visual display, not text extraction. Tables, multi-column layouts, headers, footers, images with text — all of these can cause basic parsers to produce garbage text.

PDF Challenge	What Goes Wrong	Solution
Tables	Rows and columns get merged into one long line of text	Use Docling, Camelot, or Unstructured.io — they detect table boundaries
Scanned PDFs (images)	No text extracted at all — it's just an image	Use OCR: Tesseract, PaddleOCR, or Azure Document Intelligence
Multi-column layout	Text from column 1 and column 2 gets interleaved	Use layout-aware parsers like Docling or PyMuPDF with blocks
Headers / Footers	Repeated header/footer text pollutes every page	Post-process: detect and strip repeated text across pages
Embedded images with text	Diagrams, flowcharts with text inside are ignored	Use multi-modal parsing (GPT-4o Vision) or extract images separately

2026 Recommendation

For production PDF parsing in 2026, use Docling (by IBM) or Unstructured.io. They handle tables, multi-column, headers/footers, and OCR out of the box. Basic PyPDFLoader is fine for simple text-only PDFs but fails on complex documents.

2.3 | Chunking — The Most Critical Decision

Why chunking matters more than anything else in RAG

What is Chunking?

Chunking is the process of splitting your loaded documents into smaller pieces of text. Each chunk becomes an independent, searchable unit in your vector database. When a user asks a question, the system finds the most relevant **chunks** — not entire documents or pages.

Why Can't We Just Store Entire Pages?

This is the most common beginner question. Here's a detailed answer:

Approach	What Happens	Problem
Store entire document as 1 vector	The embedding represents the AVERAGE meaning of the whole document	Too generic — a 200-page doc's vector matches everything vaguely, nothing precisely
Store each page as 1 vector	Each page's embedding represents the average meaning of that page	Pages often cover 3-4 topics — the embedding is a blend of all, matching none well
Store each chunk (300-600 chars)	Each chunk's embedding represents one specific topic/paragraph	PRECISE — when user asks about annual leave, the chunk about annual leave scores highest

The Goldilocks Problem

Chunk Size — Too Small vs Too Big vs Just Right

TOO SMALL (50-100 chars):
'All full-time employees are entitled'
→ No context! What are they entitled to? The chunk is meaningless alone.
→ The embedding can't capture any useful meaning from such a short fragment.

TOO BIG (2000-5000 chars):
'[3 paragraphs about leave policy + 2 paragraphs about sick leave + 1 paragraph about holidays]'
→ Too much noise! The embedding blends all topics together.
→ User asks about leave, but chunk also contains irrelevant holiday info.

JUST RIGHT (300-600 chars):
'All full-time employees are entitled to 24 days of annual leave per calendar year. Leave must be applied at least 5 working days in advance through the HR portal.'
→ One complete thought. Clear topic. Meaningful embedding. Perfect for retrieval.

2.4 | 7 Chunking Strategies Compared

Every method explained with pros, cons & when to use

Strategy 1: Fixed-Size Chunking

The simplest approach: split text every N characters regardless of content boundaries. Doesn't care about sentences, paragraphs, or meaning — just counts characters and cuts.

Aspect	Details
How it works	Split every N characters (e.g., every 500 chars). Add overlap of M chars between chunks.
Best for	Quick prototyping, simple text documents, when you just need something working fast.
Weakness	Cuts mid-sentence, mid-word, mid-table. Loses context at every boundary.
Example	'All employees are entitled to 24 da|ys of annual leave' — cuts at char 500, splitting 'days'.

Strategy 2: Recursive Character Splitting (Recommended Default)

The **most popular** and generally **best default** strategy. Tries to split at natural boundaries in a priority order: paragraphs first, then sentences, then words, then characters as a last resort.

Aspect	Details
How it works	Tries to split at '\n\n' (paragraph breaks) first. If chunk is still too big, tries '\n' (line breaks). Then '. ' (sentences). Then ' ' (words). Then '' (characters) as last resort.
Best for	Most text documents — PDFs, DOCX, articles, policies, manuals. This is your default choice.
Strength	Preserves natural text boundaries. Paragraphs stay together. Sentences aren't cut mid-way.
Weakness	May still split tables or structured data poorly. Doesn't understand topic changes.

This Is Your Default — Use It Until You Have a Reason Not To

RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=50 is the starting point for 90% of RAG projects. Only switch to other strategies when you have a specific problem that recursive splitting doesn't solve.

Strategy 3: Sentence-Based Splitting

Uses NLP libraries (NLTK or spaCy) to detect sentence boundaries and split at them. Each chunk contains complete sentences — never cuts mid-sentence.

Aspect	Details
How it works	NLP sentence tokenizer detects sentence boundaries. Groups of N sentences = 1 chunk.

Aspect	Details
Best for	Clean prose text — articles, blog posts, news content. Text without tables or code.
Weakness	Variable chunk sizes (some sentences are 10 words, others are 100). May produce very uneven chunks.

Strategy 4: Semantic Chunking

The smartest but most expensive approach. Uses embeddings to detect when the **topic changes** within a document and splits at those topic boundaries.

Aspect	Details
How it works	Embed each sentence. Compare consecutive sentence embeddings. When cosine similarity drops below a threshold (topic change detected), split there.
Best for	Long, complex documents with multiple topics. Research papers, legal documents, technical manuals.
Strength	Chunks represent coherent topics. Best possible retrieval quality.
Weakness	VERY slow (needs to embed every sentence). Expensive for large document sets. 10-50x slower than recursive.

Strategy 5: Token-Based Splitting

Splits based on LLM tokens (not characters). Uses the same tokenizer as the target LLM (e.g., tiktoken for GPT models). Ensures chunks fit exactly within the LLM's context window.

Aspect	Details
How it works	Count tokens using the LLM's tokenizer (tiktoken for GPT, sentencepiece for Llama). Split every N tokens.
Best for	When you need exact control over how many tokens each chunk uses. Optimizing LLM context window usage.
Weakness	Requires tokenizer library. Different LLMs use different tokenizers. May still cut mid-sentence.

Strategy 6: Markdown / HTML Header Splitting

Splits at document structure boundaries — headings (#, ##, h1, h2), sections, dividers. Each chunk corresponds to one section of the document.

Aspect	Details
How it works	Detect heading boundaries (# H1, ## H2 for Markdown; , for HTML). Each section becomes one chunk with the heading as metadata.
Best for	Wiki pages, structured documentation, README files, Confluence/Notion pages.
Strength	Chunks are naturally organized by topic. Heading becomes searchable metadata.
Weakness	Only works with structured documents. Unstructured text (PDFs, emails) won't have headings.

Strategy 7: Parent-Child Chunking (Advanced)

A two-level chunking approach: **small child chunks** for precise retrieval, **large parent chunks** for full context in the LLM prompt. You search against child chunks, but return the parent chunk to the LLM.

Aspect	Details
How it works	Create large parent chunks (2000 chars). Split each parent into small child chunks (200 chars). Index child chunks in vector DB. When a child matches, return its parent to the LLM.
Best for	When you need both precision (small chunks match better) AND context (LLM needs full paragraph).
Strength	Best of both worlds: precise retrieval + rich context for generation.
Weakness	More complex to implement. Requires a separate docstore for parents + vector DB for children.

Quick Comparison Table

Strategy	Speed	Quality	Complexity	When to Use
Fixed Size	Very Fast	Low	Very Low	Quick prototype only
Recursive (Default)	Fast	Good	Low	Most projects — start here
Sentence	Fast	Good	Low	Clean prose text
Semantic	Very Slow	Excellent	High	Complex multi-topic docs
Token-based	Fast	Medium	Medium	Exact context window control
Markdown/HTML	Fast	Good	Low	Structured wiki/docs
Parent-Child	Medium	Excellent	High	Production systems needing precision + context

2.5 | Chunking Golden Rules

Rules you must memorize — these make or break RAG quality

- **Rule 1: chunk_size = 300-600 characters**

  This is the sweet spot for most use cases. 300 chars ≈ 75 words ≈ 3-4 sentences. Enough context for a meaningful embedding, small enough for precise retrieval. Start with 500 and tune from there.

- **Rule 2: chunk_overlap = 10-15% of chunk_size**

  For chunk_size=500, use overlap=50-75. This ensures sentences at chunk boundaries are present in both chunks. Without overlap, you WILL lose information at every boundary — guaranteed.

- **Rule 3: SAME embedding model for indexing AND querying**

  This is the #1 beginner mistake. If you embed chunks with Model A but embed queries with Model B, the vectors live in completely different spaces. Similarity search becomes meaningless. ALWAYS use the same model.

- **Rule 4: Always attach metadata**

  Every chunk MUST have metadata: source filename, page number, section heading, document date, category. Without metadata, you can't filter, can't cite sources, and can't debug retrieval problems.

- **Rule 5: Test with real queries**

  The only way to know if your chunking is good: run 20-50 real questions against your indexed data. Check if the retrieved chunks actually contain the answer. Use RAGAS evaluation (Class 6).

- **Rule 6: Don't mix document types in one chunk**

  If a page has a table AND paragraph text, don't let them end up in the same chunk. Parse tables separately (as structured data) and paragraphs separately (as text).

2.6 | Chunking Example Walkthrough

Real HR document chunked step by step

Original Document: HR Policy, Page 12

Raw Text (Page 12 — 950 characters total)

Section 3.2 — Annual Leave

All full-time employees are entitled to 24 days of annual leave per calendar year. Leave must be applied at least 5 working days in advance through the HR portal. Unused leave can be carried forward for up to 90 days into the next calendar year. After 90 days, unused leave expires automatically.

Section 3.3 — Medical Leave

Medical leave entitlement is 12 days per year with a valid doctor's certificate. For absences exceeding 3 consecutive days, a medical certificate must be submitted within 48 hours of returning to work. Medical leave cannot be carried forward.

Section 3.4 — Maternity & Paternity Leave

Female employees are entitled to 180 days of maternity leave. Male employees are entitled to 15 days of paternity leave. Both must be applied at least 30 days before the expected date.

After Chunking (chunk_size=400, overlap=50)

Chunk #	Content (shortened)	Chars	Metadata
Chunk 1	Section 3.2 — Annual Leave. All full-time employees are entitled to 24 days of annual leave per calendar year. Leave must be applied at least 5 working days in advance through the HR portal. Unused leave can be carried forward for up to 90 days into the next calendar year. After 90 days, unused leave expires automatically.	~340	source: hr_policy.pdf page: 12 section: 3.2
Overlap	'...After 90 days, unused leave expires automatically. Section 3.3 — Medical Leave...'	~50	(shared between Chunk 1 and Chunk 2)
Chunk 2	Section 3.3 — Medical Leave. Medical leave entitlement is 12 days per year with a valid doctor's certificate. For absences exceeding 3 consecutive days, a medical certificate must be submitted within 48 hours of returning to work. Medical leave cannot be carried forward.	~320	source: hr_policy.pdf page: 12 section: 3.3
Overlap	'...Medical leave cannot be carried forward. Section 3.4 — Maternity...'	~50	(shared between Chunk 2 and Chunk 3)

Chunk #	Content (shortened)	Chars	Metadata
Chunk 3	Section 3.4 — Maternity & Paternity Leave. Female employees are entitled to 180 days of maternity leave. Male employees are entitled to 15 days of paternity leave. Both must be applied at least 30 days before the expected date.	~260	source: hr_policy.pdf page: 12 section: 3.4

Notice: Each chunk covers ONE section completely. The overlap ensures no information is lost at section boundaries.

2.7 | Embeddings — Text to Numbers

How text becomes searchable vectors

What is an Embedding?

An **embedding** converts text into a list of numbers (a **vector**) that captures its **semantic meaning**. An embedding model is a neural network trained on billions of text pairs to learn which texts have similar meaning. The output is a dense vector — typically 768 to 3072 floating-point numbers.

The magic property: texts with **similar meaning** produce vectors that are **geometrically close** to each other in high-dimensional space. This is what makes semantic search possible.

How Embeddings Are Created (Simplified)

Step	What Happens	Example
1. Tokenize	Text is broken into tokens (subwords)	'Annual leave' → ['Annual', 'leave']
2. Encode	Each token passes through neural network layers that understand context	The model understands 'leave' means 'time off' not 'depart' based on surrounding words
3. Pool	All token vectors are combined into one vector representing the full text	768-3072 numbers that capture the MEANING of 'Annual leave entitlement is 24 days'
4. Normalize	Vector is normalized to unit length (for cosine similarity)	[0.023, -0.891, 0.445, ...] → all vectors on same scale

Why Embeddings Beat Keywords — The Power of Meaning

User Query	Relevant Document Text	Same Words?	Embedding Finds It?
'vacation days'	'Annual leave entitlement is 24 days per year'	NO — zero words match	YES — same meaning!
'time off work'	'Employee absence policy and paid leave'	NO — 'time off' ≠ 'absence'	YES — related concept
'how many holidays'	'24 days of annual leave per calendar year'	NO — 'holidays' ≠ 'leave'	YES — similar meaning
'sick leave rules'	'Medical leave requires doctor certificate'	PARTIAL — 'leave' matches	YES — both about medical absence
'Python loops'	'Annual leave policy for employees'	NO	NO — correctly identifies as unrelated

This is why vector search is revolutionary — it understands MEANING, not just matching words.

2.8 | How Similarity Search Works

Cosine similarity, dot product & euclidean — explained simply

When a user asks a question, the system embeds the query and compares it against every stored chunk vector. The comparison uses **similarity metrics** — mathematical formulas that measure how 'close' two vectors are.

Metric	What It Measures	Range	When to Use
Cosine Similarity	The angle between two vectors. Same direction = similar meaning. Ignores vector length.	0 to 1 (0=unrelated, 1=identical)	DEFAULT for text. Use this 95% of the time.
Dot Product	Sum of element-wise multiplication. Combines direction AND magnitude.	-∞ to +∞ (higher=more similar)	When vectors are already normalized (same as cosine).
Euclidean Distance	Straight-line distance between two vector endpoints. Smaller = more similar.	0 to ∞ (0=identical, larger=different)	When magnitude matters. Less common for text.

Cosine Similarity — The One You Need to Know

Simple Analogy

Imagine two arrows pointing from the center of a circle.

- If both arrows point in the SAME direction → cosine similarity = 1.0 (identical meaning)
- If arrows are at 90° to each other → cosine similarity = 0.0 (unrelated)
- If arrows point in OPPOSITE directions → cosine similarity = -1.0 (opposite meaning)

For text embeddings:
- Score > 0.80 → Very relevant (almost certainly the right chunk)
- Score 0.65-0.80 → Relevant (worth including in top-K)
- Score 0.50-0.65 → Somewhat related (may or may not help)
- Score < 0.50 → Probably irrelevant (discard)

Rule of thumb: Set your threshold at 0.70+ for production systems.

2.9 | Embedding Models Compared (2026)

All major models with quality, cost & dimensions

Paid API Models

Model	Provider	Dimensions	Quality	Cost	Best For
text-embedding-3-small	OpenAI	1536	★★★★■	~$0.02/1M tokens	Default paid choice — good quality, affordable
text-embedding-3-large	OpenAI	3072	★★★★★	~$0.13/1M tokens	When quality matters most, budget allows
embed-english-v4.0	Cohere	1024	★★★★■	~$0.10/1M tokens	Good quality, supports 100+ languages
voyage-3-large	Voyage AI	1024	★★★★★	~$0.12/1M tokens	Top-tier quality, especially for code + text
Gemini Embedding	Google	768	★★★★■	Free tier available	If already in Google Cloud ecosystem

Free / Open-Source Models (Run Locally)

Model	Provider	Dimensions	Quality	Speed	Best For
all-MiniLM-L6-v2	HuggingFace	384	★★★■■	Very Fast	Quick prototyping, learning, small projects
BAAI/bge-large-en-v1.5	BAAI	1024	★★★★■	Medium	High quality free alternative to OpenAI
nomic-embed-text-v1.5	Nomic AI	768	★★★★■	Fast	Good balance of quality and speed
BAAI/bge-m3	BAAI	1024	★★★★★	Slow	Best multilingual — 100+ languages
Qwen3-Embedding-8B	Alibaba	Variable	★★★★★	Slow	Latest 2026 SOTA — very high quality
mxbai-embed-large-v1	MixedBread	1024	★★★★★	Medium	Top MTEB benchmark scorer

Which Embedding Model Should You Choose?

Just starting / learning → all-MiniLM-L6-v2 (free, fast, easy)
Building a real project → text-embedding-3-small (OpenAI, best cost/quality)
Need highest quality → Qwen3-Embedding-8B or voyage-3-large
Need multilingual → BAAI/bge-m3 (100+ languages, free)
No budget at all → nomic-embed-text + Ollama (100% local, $0 cost)

CRITICAL: Once you choose a model, NEVER change it without re-indexing ALL your documents.

2.10 | Vector Databases

What they do & how they're different from SQL databases

What is a Vector Database?

A **vector database** is a specialized database built for one purpose: given a query vector, find the most **similar** stored vectors as fast as possible. Traditional databases search by exact match (WHERE name = 'John'). Vector databases search by **similarity** ('find vectors closest to this one').

Vector DB vs Traditional DB

Aspect	Traditional DB (SQL/NoSQL)	Vector Database
Search type	Exact match: WHERE price > 100	Similarity: find nearest vectors to query
Data stored	Rows and columns, documents, key-values	High-dimensional vectors + original text + metadata
Query example	SELECT * FROM products WHERE name LIKE '%shoe%'	Find top-5 vectors most similar to embed('comfortable running shoes')
Speed at scale	Fast for indexed columns	Fast for vector similarity (uses ANN algorithms)
Indexing method	B-tree, hash index	HNSW, IVF, PQ (approximate nearest neighbor)
Use case	Business data, transactions, CRUD	Semantic search, RAG, recommendation, image search

How Vector DBs Achieve Speed — ANN Algorithms

Searching millions of vectors by comparing against every single one (brute force) would be too slow. Vector databases use **Approximate Nearest Neighbor (ANN)** algorithms that trade a tiny bit of accuracy for massive speed improvements — typically 99%+ accuracy with 100x+ speed.

Algorithm	How It Works (Simplified)	Used By
HNSW (Hierarchical Navigable Small World)	Builds a multi-level graph. Starts at top level (coarse), navigates down to find nearest neighbors. Like finding a house by first finding the country, then city, then street, then address.	Pinecone, Qdrant, Weaviate, ChromaDB
IVF (Inverted File Index)	Groups vectors into clusters using k-means. At query time, only searches the nearest clusters instead of all vectors. Like searching only the relevant section of a library.	FAISS, Milvus

Algorithm	How It Works (Simplified)	Used By
PQ (Product Quantization)	Compresses vectors into smaller representations. Loses some accuracy but uses 10-100x less memory. Like storing a thumbnail instead of the full image.	FAISS (IVF_PQ), Milvus

2.11 | Vector DB Comparison (2026)

All major options compared — choose the right one for your project

Database	Type	Hosting	Scale	Filtering	Best For
ChromaDB	Open Source	Local + Cloud	Millions	Basic metadata	Learning, prototyping, small apps. Easiest setup — pip install chromadb and done.
FAISS (by Meta)	Open Source Library	Local only (in-memory)	Billions	None built-in	Speed-critical research applications. Blazing fast but no persistence, no metadata filtering.
Pinecone	Fully Managed SaaS	Cloud only	Billions	Excellent	Production apps with zero maintenance. Most popular managed option in 2026. Serverless pricing available.
Qdrant	Open Source	Local + Cloud	Billions	Excellent	Advanced filtering + vector search. Rising star in 2026. Rust-based = very fast. Great for self-hosted production.
Weaviate	Open Source	Local + Cloud	Billions	Excellent	GraphQL interface + hybrid search (vector + keyword). Good for complex query patterns.
Milvus	Open Source	Local + Cloud	Trillions	Good	Enterprise-scale massive datasets. LinkedIn, Shopee, and other large companies use it.
pgvector	PostgreSQL Extension	Any Postgres	Millions	Full SQL	Already using PostgreSQL? Just add an extension. No new infrastructure needed.

Which Vector DB to Choose? (2026 Decision Guide)

Just learning / prototyping → ChromaDB (zero config, pip install, done)
Small production app → Qdrant (self-hosted, free, excellent quality)
Production with zero maintenance → Pinecone (managed, serverless pricing)
Already use PostgreSQL → pgvector (just add extension)
Need extreme speed → FAISS (in-memory, fastest, but no persistence)
Enterprise / billions of vectors → Milvus or Pinecone Enterprise
Complex filtering + search → Qdrant or Weaviate

Vector DB market in 2026: $2.12 billion (up 30.5% from 2025). This is a fast-growing space.

2.12 | Putting It All Together

Complete indexing pipeline example — HR Policy PDF

Example: Indexing AmanCorp's HR Policy

Scenario

Document: hr_policy_2024.pdf (200 pages, 50MB)
Goal: Make it searchable for an employee Q&A; chatbot
Stack: LangChain + OpenAI Embeddings + ChromaDB

Step	What Happens	Numbers
1. Load Document	PyPDFLoader reads the PDF, extracts text from each page with page number metadata	Input: 1 PDF file Output: 200 Document objects (one per page)
2. Chunk Text	RecursiveCharacterTextSplitter splits pages into 500-char chunks with 50-char overlap	Input: 200 pages Output: ~847 chunks Avg chunk: ~470 chars
3. Add Metadata	Each chunk gets enriched metadata: source file, page number, section heading (if detected)	Each chunk now has: • source: hr_policy_2024.pdf • page: 1-200 • chunk_id: 0-846
4. Create Embeddings	OpenAI text-embedding-3-small converts each chunk into a 1536-dimension vector	Input: 847 text chunks Output: 847 vectors (1536 floats each) Time: ~2-3 minutes Cost: ~$0.01
5. Store in Vector DB	ChromaDB stores all vectors + original text + metadata. Builds HNSW index for fast search.	Input: 847 vectors + text + metadata Output: Searchable vector DB DB size: ~15MB on disk

Result

After these 5 steps (which take about 5 minutes total), you have a fully indexed, searchable knowledge base. Any employee can now ask questions like 'How many leaves do I get?' and the system will find the exact relevant chunk in milliseconds.

This indexing runs ONCE. After that, Phase 2 (Querying) handles every user question in real-time.

Minimal Code — For Reference Only

```
// Complete Indexing Pipeline (reference)

# Complete indexing in ~15 lines
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

docs = PyPDFLoader('hr_policy_2024.pdf').load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
).split_documents(docs)
vectordb = Chroma.from_documents(
    documents=chunks,
    embedding=OpenAIEmbeddings(model='text-embedding-3-small'),
    persist_directory='./chroma_db'
)
print(f'Indexed {len(chunks)} chunks!')
```

2.13 | Common Indexing Mistakes

Top 8 failures with exact fixes

#	Mistake	What Goes Wrong	How to Fix
1	chunk_size too large (2000+)	Chunks contain multiple topics. Embedding is a blend of all topics — matches everything vaguely, nothing precisely.	Reduce to 300-600 chars. Test with real queries.
2	chunk_size too small (50-100)	Chunks have no context. 'employees are entitled' means nothing alone. Embeddings are meaningless.	Increase to 300-600 chars minimum.
3	No chunk overlap	Sentences at boundaries get cut in half. 'Leave can be carried forward for 90' | 'days into the next year.' — split mid-fact.	Set overlap = 10-15% of chunk_size.
4	Different embedding model for index vs query	Vectors live in different mathematical spaces. Cosine similarity returns random results.	Use the EXACT same model for both. Never change without re-indexing everything.
5	No metadata on chunks	Can't filter by source, can't cite page numbers, can't debug why wrong chunks are retrieved.	Always add: source file, page number, section, date, category.
6	Using basic PDF parser for complex PDFs	Tables become gibberish. Multi-column text gets interleaved. Headers/footers pollute every chunk.	Use Docling or Unstructured.io for complex PDFs.
7	Not cleaning text before chunking	Extra whitespace, HTML tags, special characters, repeated headers — all create noise in embeddings.	Strip HTML, normalize whitespace, remove headers/footers before chunking.
8	Never evaluating indexing quality	No idea if chunks are correct until production users complain.	Run RAGAS Context Precision and Context Recall on 20-50 test questions.

2.14 | Summary & What's Next

Key takeaways + preview of Class 3

Class 2 — Key Takeaways

- **Indexing = Load** → **Chunk** → **Embed** → **Store.** This runs once (or when docs change). It's the foundation of RAG.
- **Document loading** extracts text + metadata from PDFs, DOCX, web pages, databases, wikis, etc.
- **Chunking is the #1 factor** affecting RAG quality. Get this wrong and nothing else matters.
- **7 chunking strategies exist** — start with RecursiveCharacterTextSplitter (chunk_size=500, overlap=50).
- **Chunk_size = 300-600 chars**. Too small = no context. Too big = noisy retrieval.
- **Embeddings convert text to vectors** that capture semantic meaning. Similar texts = close vectors.
- **Cosine similarity** is the default metric. Score > 0.70 = relevant. < 0.50 = irrelevant.
- **Use the SAME embedding model** for indexing and querying — the #1 rule, never break it.
- **Vector databases** (ChromaDB, Pinecone, Qdrant) enable millisecond similarity search across millions of vectors.
- **Metadata is essential** — always add source, page, section, date to every chunk.

Preview of Class 3: Retrieval Techniques

What We'll Cover in Class 3

Now that your documents are indexed, how do you FIND the right chunks? Class 3 covers:

• Dense Retrieval — semantic vector search in depth
• Sparse Retrieval — BM25 keyword matching and when it's better
• Hybrid Retrieval — combining dense + sparse (production best practice)
• Reranking — using cross-encoders to improve precision by 20-40%
• MMR — Maximum Marginal Relevance for diverse results
• Metadata Filtering — narrowing search before vector comparison
• Query Rewriting — fixing vague queries before searching

Make sure you've understood Indexing deeply before moving to Retrieval!

6-Class Roadmap

Class	Title	Status
Class 1	Foundation — What, Why & How RAG Works	■ Completed
Class 2 (TODAY)	Indexing Pipeline — Chunking, Embeddings, Vector DB	■ Completed
Class 3	Retrieval Techniques — Dense, Sparse, Hybrid, Reranking	■ Next

Class	Title	Status
Class 4	Generation & Prompt Design — Chains, Citations, Full Build	■ Coming
Class 5	Advanced RAG — HyDE, CRAG, Self-RAG, GraphRAG, Agentic	■ Coming
Class 6	Production & Interview Prep — RAGAS, Deployment, 20 Q&A;	■ Coming